In [ ]:
```
# Cell 1: Install Hugging Face Transformers and PyTorch
!pip install -q transformers datasets torch
print("Libraries installed successfully!")
```

```
Libraries installed successfully!
```

In [ ]:
```
# Cell 2 (Updated): Import dependencies and load GPT-2 with a padding tok
from transformers import GPT2LMHeadModel, GPT2Tokenizer
import torch

# Load tokenizer and model
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')

# FIX: Set the padding token for GPT-2
tokenizer.pad_token = tokenizer.eos_token

model = GPT2LMHeadModel.from_pretrained('gpt2')

# Set device to GPU if available
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)

print(f"GPT-2 Model and Tokenizer loaded successfully on: {device}")
```

```
GPT-2 Model and Tokenizer loaded successfully on: cpu
```

In [ ]:
```
# Cell 3: Generate coherent text based on a prompt
prompt = "The future of artificial intelligence will involve"

# Encode the input prompt into tokens and move to GPU
input_ids = tokenizer.encode(prompt, return_tensors='pt').to(device)

# Generate text using sampling parameters
output_ids = model.generate(
    input_ids,
    max_length=150,          # Maximum length of the generated text
    num_return_sequences=1,  # Number of responses to generate
    no_repeat_ngram_size=2,  # Prevents repeating word phrases
    temperature=0.7,         # Controls creativity (higher = more creativ
    top_k=50,                # Limits vocabulary choices
    top_p=0.95,              # Nucleus sampling threshold
    do_sample=True
)

# Decode tokens back into readable text
generated_text = tokenizer.decode(output_ids[0], skip_special_tokens=True

print("=" * 50)
print("PROMPT:", prompt)
print("=" * 50)
print("GENERATED TEXT:\n", generated_text)
```

```
==================================================
PROMPT: The future of artificial intelligence will involve
==================================================
GENERATED TEXT:
 The future of artificial intelligence will involve both humans and robot
s, but the most likely outcome is that we will see a transition to AI fro
m a near-future to a more human-like world.

The next step is to create a system that can run on humans, without the n
eed for robots. In this case, the human AI will be able to run off humans
(as well as many robots) but not AI controlled by robots (like in the cas
e of the future.)
. . .
- The human, or in some cases, AI
 (and machine) will become more complex than humans in many ways. The bas
ic idea is this. If you want a new kind of computer, you will need to dev
elop algorithms to help
```

In [ ]:

```python
# Cell 4 (Fixed): Proper Dataset Size for Training
from transformers import DataCollatorForLanguageModeling, Trainer, Traini
from datasets import Dataset

# 1. Create a larger sample training text file with more sentences
dataset_filename = "my_custom_data.txt"
with open(dataset_filename, "w", encoding="utf-8") as f:
    f.write("Artificial intelligence and machine learning transform indus
    f.write("Data science combines statistics, programming, and domain kn
    f.write("Neural networks mimic human brain structures to solve comple

# 2. Load data using the Hugging Face 'datasets' library
raw_dataset = Dataset.from_text(dataset_filename)

# 3. Tokenize the text data
def tokenize_function(examples):
    return tokenizer(examples["text"], truncation=True, max_length=128)

tokenized_datasets = raw_dataset.map(
    tokenize_function,
    batched=True,
    remove_columns=["text"]
)

# 4. Group texts into smaller chunks for language modeling
block_size = 16  # Reduced block size so it fits our text sample
def group_texts(examples):
    concatenated_examples = {k: sum(examples[k], []) for k in examples}
    total_length = len(concatenated_examples[list(examples.keys())[0]])
    total_length = (total_length // block_size) * block_size
    result = {
        k: [t[i : i + block_size] for i in range(0, total_length, block_s
        for k, t in concatenated_examples.items()
    }
    result["labels"] = result["input_ids"].copy()
    return result
```

```python
train_dataset = tokenized_datasets.map(
    group_texts,
    batched=True,
)

# 5. Setup Data Collator
data_collator = DataCollatorForLanguageModeling(
    tokenizer=tokenizer,
    mlm=False # False because GPT-2 uses Causal Language Modeling
)

# 6. Define training arguments
training_args = TrainingArguments(
    output_dir="./gpt2-finetuned",
    num_train_epochs=3,
    per_device_train_batch_size=2,
    save_steps=10,
    logging_steps=5,
    learning_rate=5e-5
)

# 7. Initialize Trainer and start fine-tuning
trainer = Trainer(
    model=model,
    args=training_args,
    data_collator=data_collator,
    train_dataset=train_dataset,
)

print("Starting fine-tuning...")
trainer.train()
print("Fine-tuning completed successfully!")
```

```
Starting fine-tuning...
[transformers] `loss_type=None` was set in the config but it is unrecogni
zed. Using the default loss: `ForCausalLMLoss`.
```

[18/18 01:46, Epoch 3/3]

Step	Training Loss
5	3.809548
10	2.395389
15	1.463218

```
/usr/local/lib/python3.13/dist-packages/torch/utils/data/dataloader.py:77
5: UserWarning: 'pin_memory' argument is set as true but no accelerator i
s found, then device pinned memory won't be used.
  super().__init__(loader)
```

```
Fine-tuning completed successfully!
```

In [ ]: